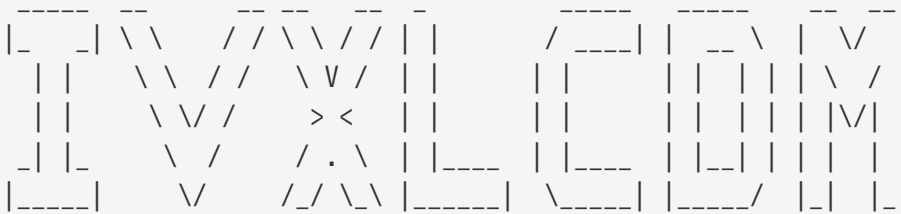


Prova pratica

Descrizione del progetto

Implementare un convertitore per i primi 1000 numeri interi da arabi a romani. Implementare un metodo per stampare i numeri romani in asciart:



Il programma deve soddisfare i requisiti descritti a [questo indirizzo](#).

Si consiglia di implementare il progetto seguendo il seguente ordine:

- Convertire e stampare i primi 3 numeri interi
- Convertire e stampare i primi 6 numeri interi
- Convertire e stampare i primi 10 numeri interi
- Convertire e stampare i primi 20 numeri interi
- Convertire e stampare i primi 50 numeri interi
- Convertire e stampare i primi 100 numeri interi
- Convertire e stampare i primi 500 numeri interi
- Convertire e stampare i primi 1000 numeri interi

Interfaccia e oggetti

Sviluppare il progetto implementando le seguenti classi.

```
package it.unipd.mtss;

public class IntegerToRoman {

    public static String convert(int number){
        // TODO
        return null;
    }
}
```

```
package it.unipd.mtss;
```

```
public class RomanPrinter {
    public static String print(int num){
        return printAsciiArt(IntegerToRoman.convert(num));
    }

    private static String printAsciiArt(String romanNumber){
        //TODO
        return null;
    }
}
```

Svolgimento

Sviluppare il progetto seguendo la pratica della Continuous Integration, pertanto si richiede di svolgere la prova nel seguente ordine:

- Predisposizione del repository github
- Definizione delle attività nel Issue Traking System
 - Suddividere le attività tra i due sviluppatori
- Creazione del progetto utilizzando l'archetipo maven quick start
- Configurazione del processo di build
- attivazione e configurazione dell'analisi statica
- Predisposizione della Build Automation con GitHub Actions
- Sviluppo del codice sorgente e dei test

Di seguito vengono specificate in dettaglio le configurazioni dei vari strumenti.

Issue Traking System

Gestire il progetto nel issue tracking system di github:

- registrare la nuova versione (vedi laboratorio ITS: Milestone)
- creare la project board (vedi laboratorio 1: Project Board)
- creare un numero congruo di attività per la realizzazione del progetto
- suddividere le attività tra i due sviluppatori

Version Control System

- Sviluppare il progetto utilizzando un repository git pubblico in Github.
- Le attività devono essere gestite utilizzando il workflow feature branch - *Git Flow*.
- Le attività devono essere sviluppate (compresi i test di unità) una per volta senza tenere in

considerazione le attività successive.

- Tutti i branch devono essere pubblicati.
- Obbligatoria almeno l'apertura e gestione di una Pull Request (revisione e merge)

Project Automation

- Creare il progetto con Maven utilizzando l'archetipo: `maven-archetype-quickstart`
- Utilizzare i seguenti parametri:
 - **groupId**: it.unipd.mtss
 - **artifactId**: roman-number

È richiesta la configurazione di una pipeline tramite GitHub Actions. La pipeline deve essere definita manualmente (non è consentito l'uso di template predefiniti) e deve includere almeno i seguenti step:

- compilazione del progetto (eseguita nella fase di compile)
- esecuzione dei test (eseguita nella fase di test)
- analisi statica del codice (eseguita nella fase di package) (es. Checkstyle o PMD)

La pipeline deve essere eseguita automaticamente ad ogni push/Pull Request e rispettare i seguenti requisiti:

- ogni step deve essere chiaramente separato e leggibile (ordinata e commentata)
- la pipeline deve fallire in caso di errori
- deve essere possibile distinguere facilmente la causa del fallimento (errore di compilazione, test falliti, violazioni di qualità)

Test di Unità

- Creare dei test di unità per soddisfare le caratteristiche **A-TRIP** e **Right Bicep** viste a lezione. È richiesta la copertura del codice $\geq 85\%$.
- I test di unità devono essere sviluppati con il framework JUnit 4 o 5 e seguire le convenzioni Maven (vedi [qui](#) e [qui](#))
- Le firme dei metodi di test devono essere parlanti. I test devono essere sviluppati seguendo il pattern [Arrange/Act/Assert \(AAA\)](#)

Analisi statica del codice

Configurare il [plugin maven checkstyle](#) in modo da effettuare le seguenti verifiche al codice di produzione (creare il file `checkstyle.xml` e configurare i seguenti moduli):

- [BooleanExpressionComplexity](#)
- [CyclomaticComplexity](#)

- [FileLength](#)
- [LineLength](#)
- [MethodLength](#)
- [EmptyCatchBlock](#)
- [FileTabCharacter](#)
- [AvoidStarImport](#)
- [IllegalImport](#)
- [NeedBraces](#)
- [Header](#) con il seguente valore (sostituire le variabili con i vostri dati)

```

////////////////////////////////////
// [NOME1] [COGNOME1] [MATRICOLA1]
// [NOME2] [COGNOME2] [MATRICOLA2]
////////////////////////////////////

```

Configurare il plugin in modo da far fallire la build se non vengono rispettati i controlli configurati.

Build Automation

- Attivare la Build Automation con GitHub Action.
- Pubblicare il risultato del processo di build nella pagina README.md del progetto
- Attivare il calcolo del code coverage tramite [Coveralls](#)
- Pubblicare il risultato dell'analisi statica e del code coverage nella pagina README.md del progetto

Modalità di consegna

- La prova deve essere **completata e consegnata entro il 11/05/2025 alle 18:00**. Ogni consegna e modifica successiva non sarà valutata.
- Il codice prodotto deve essere rilasciato nel ramo **master** o **main**
- La consegna deve essere fatta tramite l'apposito form che verrà pubblicato nel sito del corso